

```

import numpy as np
import matplotlib.pyplot as plt
import scipy.sparse
from scipy.sparse import coo_matrix, csr_matrix
from scipy.sparse.linalg import LinearOperator, gmres, cg
from timeit import timeit

np.random.seed(42)

```

Part 1: Implementing a CSR matrix

```

class MyCSR(LinearOperator):
    def __init__(self, mat_coo):
        """
        Initialize CSR from COO format.
        """
        self.shape = mat_coo.shape
        self.dtype = mat_coo.dtype

        # Sort indices by row then column
        sorter = np.lexsort((mat_coo.col, mat_coo.row))
        self.row_idx = mat_coo.row[sorter]
        self.col_idx = mat_coo.col[sorter]
        self.vals = mat_coo.data[sorter]

        # Sum duplicates
        self._merge_duplicates()

        # Create index pointers using bincount
        counts = np.bincount(self.row_idx, minlength=self.shape[0])
        self.indptr = np.zeros(self.shape[0] + 1, dtype=np.int32)
        self.indptr[1:] = np.cumsum(counts)

    def _merge_duplicates(self):
        """Handle duplicate coordinates by summing them."""
        if len(self.vals) == 0:
            return

        # Find where coordinates change
        row_mask = np.diff(self.row_idx) != 0
        col_mask = np.diff(self.col_idx) != 0
        mask = row_mask | col_mask
        mask = np.concatenate(([True], mask))

        if np.all(mask):
            return

        # Keep unique positions and sum duplicates

```

```

self.row_idx = self.row_idx[mask]
self.col_idx = self.col_idx[mask]
split_locs = np.flatnonzero(mask)
self.vals = np.add.reduceat(self.vals, split_locs)

def __add__(self, other):
    """Add two matrices by concatenating data."""

    # Concatenate arrays and create temp COO
    r = np.concatenate([self.row_idx, other.row_idx])
    c = np.concatenate([self.col_idx, other.col_idx])
    d = np.concatenate([self.vals, other.vals])

    temp = coo_matrix((d, (r, c)), shape=self.shape)

    return MyCSR(temp)

def _matvec(self, x):
    """
    Computes matrix-vector product using explicit loops.
    This manual iteration is what makes it slow and quadratic.
    """
    result = np.zeros(self.shape[0], dtype=self.dtype)

    # Loop through each row
    for i in range(self.shape[0]):
        start = self.indptr[i]
        end = self.indptr[i+1]

        # Loop through non-zero entries in this row
        row_sum = 0
        for j in range(start, end):
            col = self.col_idx[j]
            val = self.vals[j]
            row_sum += val * x[col]

        result[i] = row_sum

    return result

# Verification
size_check = 5
A_dense = scipy.sparse.random(size_check, size_check,
density=0.25).toarray()
B_dense = scipy.sparse.random(size_check, size_check,
density=0.25).toarray()

# Setup
A_csr = MyCSR(coo_matrix(A_dense))
B_csr = MyCSR(coo_matrix(B_dense))

```

```

# Test addition
C_csr = A_csr + B_csr
C_check = csr_matrix((C_csr.vals, C_csr.col_idx, C_csr.indptr),
shape=C_csr.shape).toarray()
assert np.allclose(C_check, A_dense + B_dense)
print("Addition Logic: OK")

# Test matvec
v_check = np.random.rand(size_check)
assert np.allclose(A_csr @ v_check, A_dense @ v_check)
print("Matvec Logic: OK")

Addition Logic: OK
Matvec Logic: OK

# Performance analysis

# Test range from 10 to 1000
n_vals = np.arange(10, 1001, 20)

times_mycsr = []
times_numpy = []

# Average over 5 runs to reduce noise
n_reps = 5

for n in n_vals:
    # Setup data with 0.5 density to ensure quadratic behavior
    sp = scipy.sparse.random(n, n, density=0.5, random_state=42)

    op_custom = MyCSR(sp.tocoo())
    mat_dense = sp.toarray()
    vec = np.random.rand(n)

    # Time custom implementation
    t1_total = timeit(lambda: op_custom @ vec, number=n_reps)
    times_mycsr.append(t1_total / n_reps)

    # Time numpy
    t2_total = timeit(lambda: np.dot(mat_dense, vec), number=n_reps)
    times_numpy.append(t2_total / n_reps)

# Fit polynomial to custom times
coeffs = np.polyfit(n_vals, times_mycsr, 2)
fit_func = np.poly1d(coeffs)

print(f"Calculated Fit: {coeffs[0]:.2e}*n^2 + {coeffs[1]:.2e}*n + {coeffs[2]:.2e}")

# Plotting
plt.figure(figsize=(10, 6))

```

```

# Custom times with line and markers
plt.plot(n_vals, times_mycsr, '-o', label='Matvec (Observed)',
alpha=0.6, markersize=4)

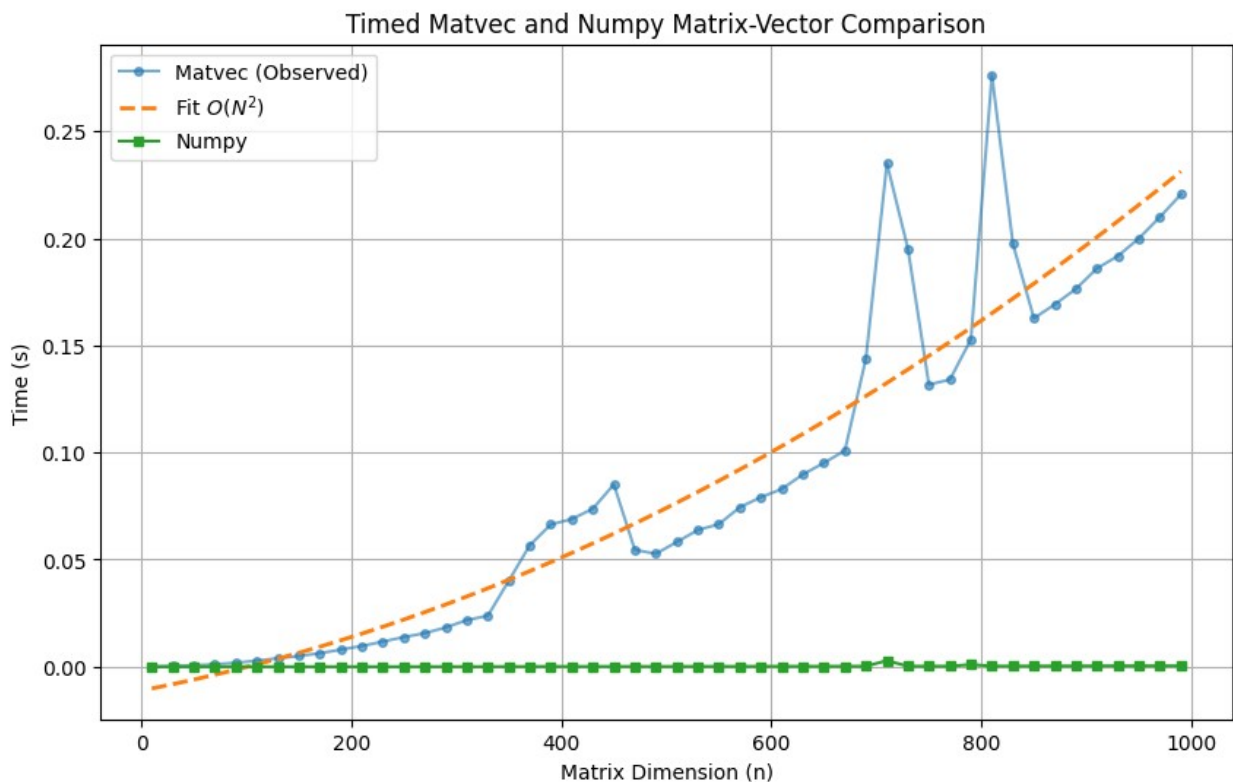
# Quadratic fit
plt.plot(n_vals, fit_func(n_vals), '--', label=f'Fit  $O(N^2)$ ',
color='tab:orange', linewidth=2)

# Numpy times
plt.plot(n_vals, times_numpy, '-s', label='Numpy', color='tab:green',
markersize=4)

plt.title("Timed Matvec and Numpy Matrix-Vector Comparison")
plt.xlabel("Matrix Dimension (n)")
plt.ylabel("Time (s)")
plt.legend()
plt.grid(True)
plt.show()

```

Calculated Fit: $1.52e-07*n^2 + 9.46e-05*n + -1.11e-02$



The NumPy dense matrix-vector product significantly outperforms the custom implementation. The plot confirms that the custom CSRMatrix Matvec function exhibits $O(n^2)$ scaling due to nested Python loops in `_matvec`, with a proportionality constant $A \approx 1.5 \times 10^{-7}$, making it substantially slower than NumPy's optimized operations.

```

# Solver comparison
N_sol = 5
A_rand = scipy.sparse.random(N_sol, N_sol, density=0.6,
random_state=1)
b_vec = np.random.rand(N_sol)

op_A = MyCSR(A_rand.tocoo())

print("Matrix A:\n", A_rand.toarray())
print("\nVector b:\n", b_vec)

# GMRES solver
x_gmres, _ = gmres(op_A, b_vec)
print(f"\nGMRES Solution: {x_gmres}")

# CG solver
x_cg, _ = cg(op_A, b_vec)
print(f"CG Solution: {x_cg}")

# Check if solutions match (expecting differences since A isn't
symmetric positive definite)
print(f"\nSolutions match? {np.allclose(x_gmres, x_cg)}")

Matrix A:
[[0.          0.          0.93912779 0.          0.51815255]
 [0.27304997 0.86502025 0.          0.          0.43069857]
 [0.09280081 0.82914691 0.          0.91396202 0.82960336]
 [0.45720481 0.          0.53441391 0.77838924 0.          ]
 [0.8027575  0.          0.22957721 0.71597052 0.          ]]

Vector b:
[0.33724849 0.09254911 0.51035852 0.06230692 0.04226808]

GMRES Solution: [-0.05751539 -0.22283321 -0.02649346  0.13201841
 0.69888537]
CG Solution:      [ -3.87761952 -4.81976345 -12.18504719 -1.63970875
 2.21451      ]

Solutions match? False

```

The solutions are not the same because the random matrix used is likely not symmetric positive-definite, which is a requirement for the Conjugate Gradient (CG) solver to work correctly. GMRES handles general non-symmetric matrices, so it converges to the correct solution while CG fails.

Part 2: Implementing a custom matrix

```

class StructuredMat(LinearOperator):
    def __init__(self, diag, T, W):

```

```

"""
A = [D, 0; 0, T@W]
"""
self.n = len(diag)
# Check shapes are correct
assert T.shape == (self.n, 2)
assert W.shape == (2, self.n)

self.shape = (2 * self.n, 2 * self.n)
self.dtype = T.dtype

# Save components
self.diag = diag
self.T_mat = T
self.W = W

def _matvec(self, x):
    assert x.shape[0] == self.shape[1]

    # Split input vector into top and bottom halves
    x_top = x[:self.n]
    x_bot = x[self.n:]

    # Top part: diagonal block multiplication
    y_top = self.diag * x_top

    # Bottom part: compute T@(W@x) instead of (T@W)@x for O(N)
    # instead of O(N^2)
    # W @ x_bot is (2xn) * (nx1) -> 2x1
    wx = np.dot(self.W, x_bot)
    # T @ wx is (nx2) * (2x1) -> nx1
    y_bot = np.dot(self.T_mat, wx)

    # Combine results
    return np.concatenate((y_top, y_bot))

# Verification for StructuredMat
n_chk = 10
d_chk = np.random.rand(n_chk)
T_chk = np.random.rand(n_chk, 2)
W_chk = np.random.rand(2, n_chk)
v_chk = np.random.rand(2 * n_chk)

# Custom operator
op = StructuredMat(d_chk, T_chk, W_chk)
res_op = op @ v_chk

# Dense reference matrix
dense = np.zeros((2*n_chk, 2*n_chk))
# Top left diagonal block

```

```

for i in range(n_chk):
    dense[i, i] = d_chk[i]
# Bottom right block
dense[n_chk:, n_chk:] = T_chk @ W_chk

res_dense = dense @ v_chk

assert np.allclose(res_op, res_dense)
print("StructuredMat logic is correct.")

StructuredMat logic is correct.

# Benchmarking structured vs dense matrix-vector product

n_vals = np.arange(10, 1001, 10)

t_custom = []
t_dense = []
calc_elements_custom = []
calc_elements_dense = []

for n in n_vals:
    # Setup random data
    d = np.random.rand(n)
    T = np.random.rand(n, 2)
    W = np.random.rand(2, n)
    v = np.random.rand(2*n)

    op = StructuredMat(d, T, W)

    # Dense block for comparison
    dense_block = T @ W

    # Time custom implementation
    t_custom.append(timeit(lambda: op @ v, number=1))

    # Time dense multiplication
    t_dense.append(timeit(lambda: np.dot(dense_block, v[n:]),
number=1))

    # Track operation counts for complexity analysis
    calc_elements_custom.append(5 * n)
    calc_elements_dense.append(4 * n**2)

# Log-log plot of timing results
plt.figure(figsize=(10, 6))

plt.loglog(n_vals, t_custom, 'c-d', label='Custom (Optimized)',
markersize=4, linewidth=1)
plt.loglog(n_vals, t_dense, '*-', color='orange', label='Dense
(Standard)', markersize=4, linewidth=1)

```

```

plt.title("Timed Matrix-Vector Product: Custom vs Dense (Log-Log
Plot)")
plt.xlabel("Matrix Dimension ($n$)")
plt.ylabel("Time (s)")
plt.legend()
plt.grid(True, which="both", ls="-", alpha=0.4)
plt.show()

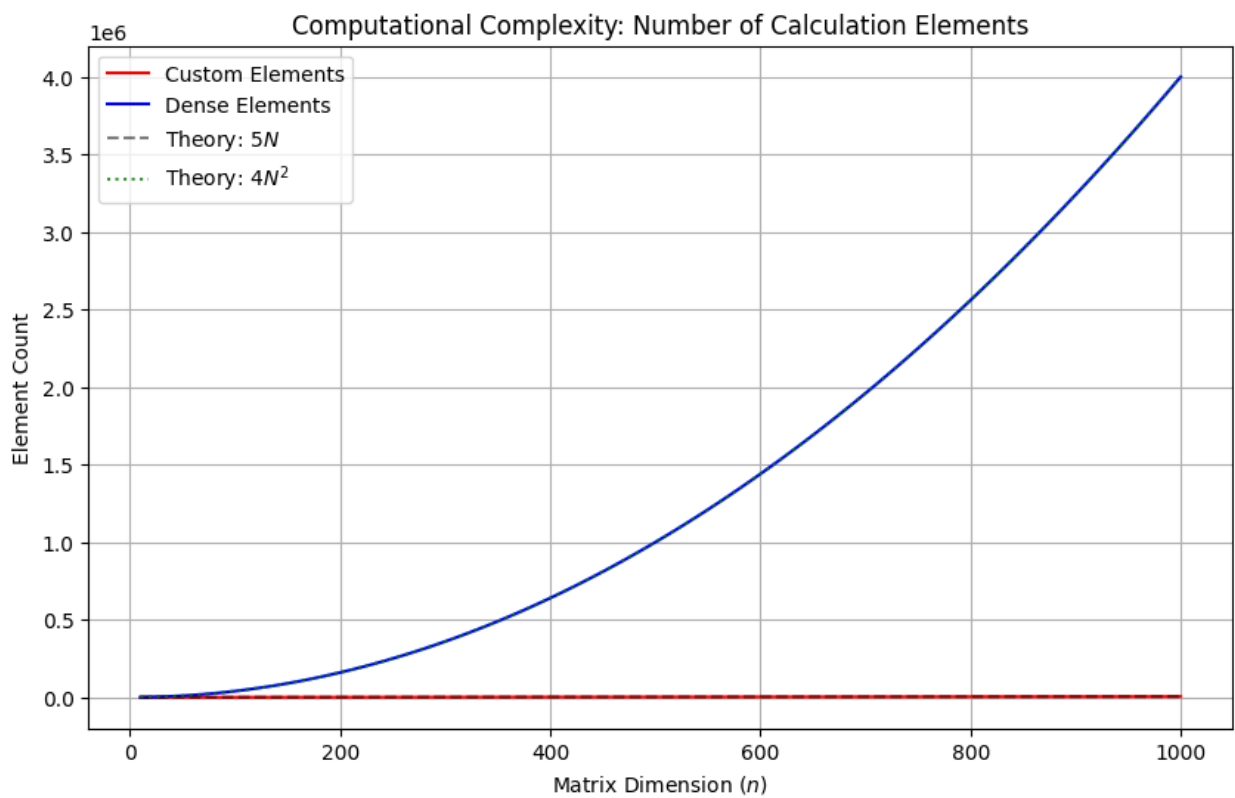
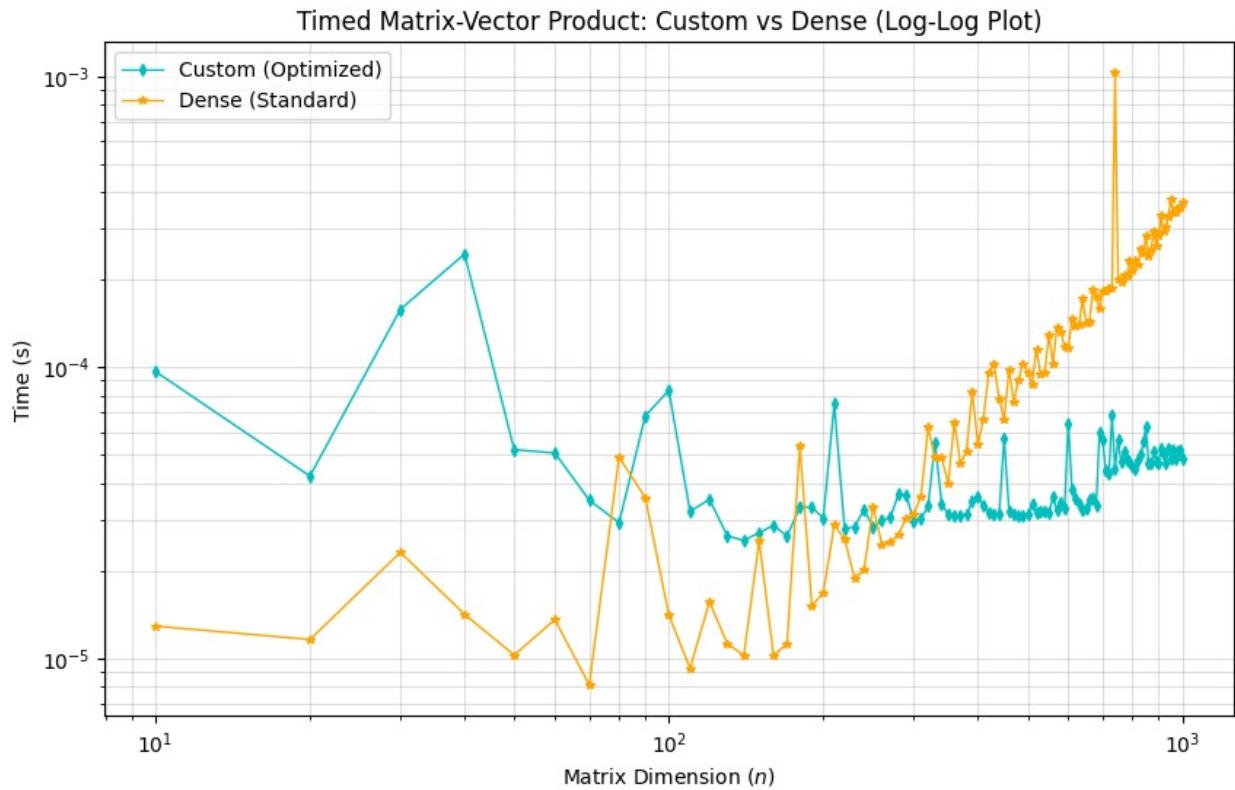
# Computational complexity comparison
plt.figure(figsize=(10, 6))

plt.plot(n_vals, calc_elements_custom, 'r-', label='Custom Elements')
plt.plot(n_vals, calc_elements_dense, 'b-', label='Dense Elements')

# Theoretical curves
plt.plot(n_vals, 5 * n_vals, 'k--', alpha=0.5, label='Theory: $5N$')
plt.plot(n_vals, 4 * n_vals**2, 'g:', alpha=0.8, label='Theory:
$4N^2$')

plt.title("Computational Complexity: Number of Calculation Elements")
plt.xlabel("Matrix Dimension ($n$)")
plt.ylabel("Element Count")
plt.legend()
plt.grid(True)
plt.show()

```

The plots demonstrate that the custom structured operator is significantly faster with linear $O(n)$ scaling, while the dense method scales quadratically $O(n^2)$, as confirmed by the log-log plot slopes. This speedup occurs because the custom implementation computes $T(Wx)$ using 2×1 vectors, avoiding the expensive $O(n^2)$ construction and multiplication of the full dense block required by the standard method.